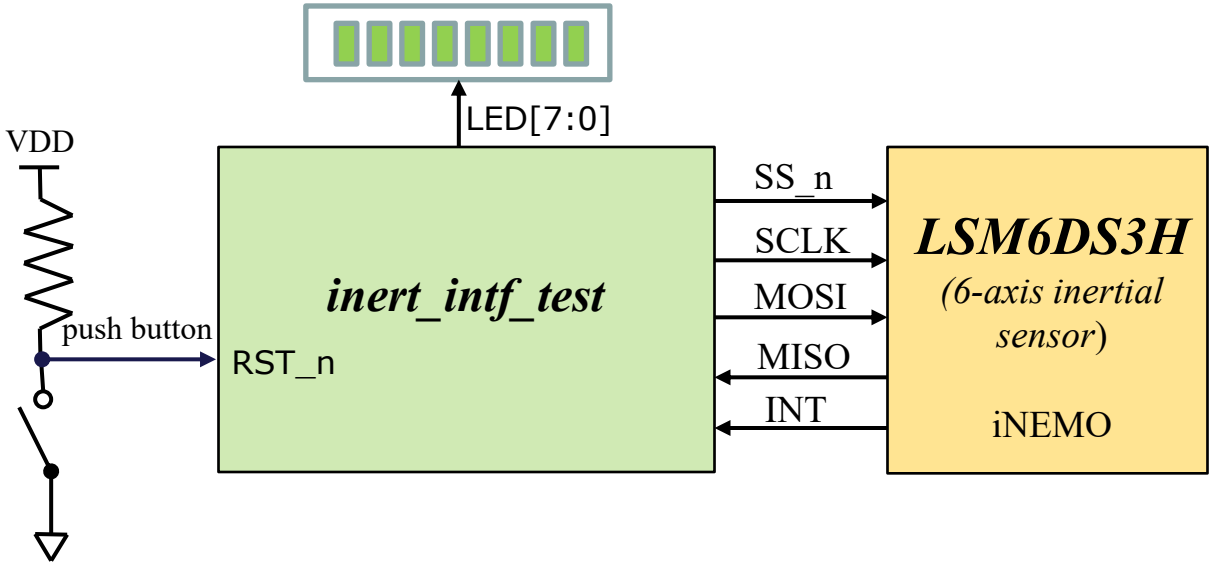
Exercise 18: Testing inert_intf on DE0

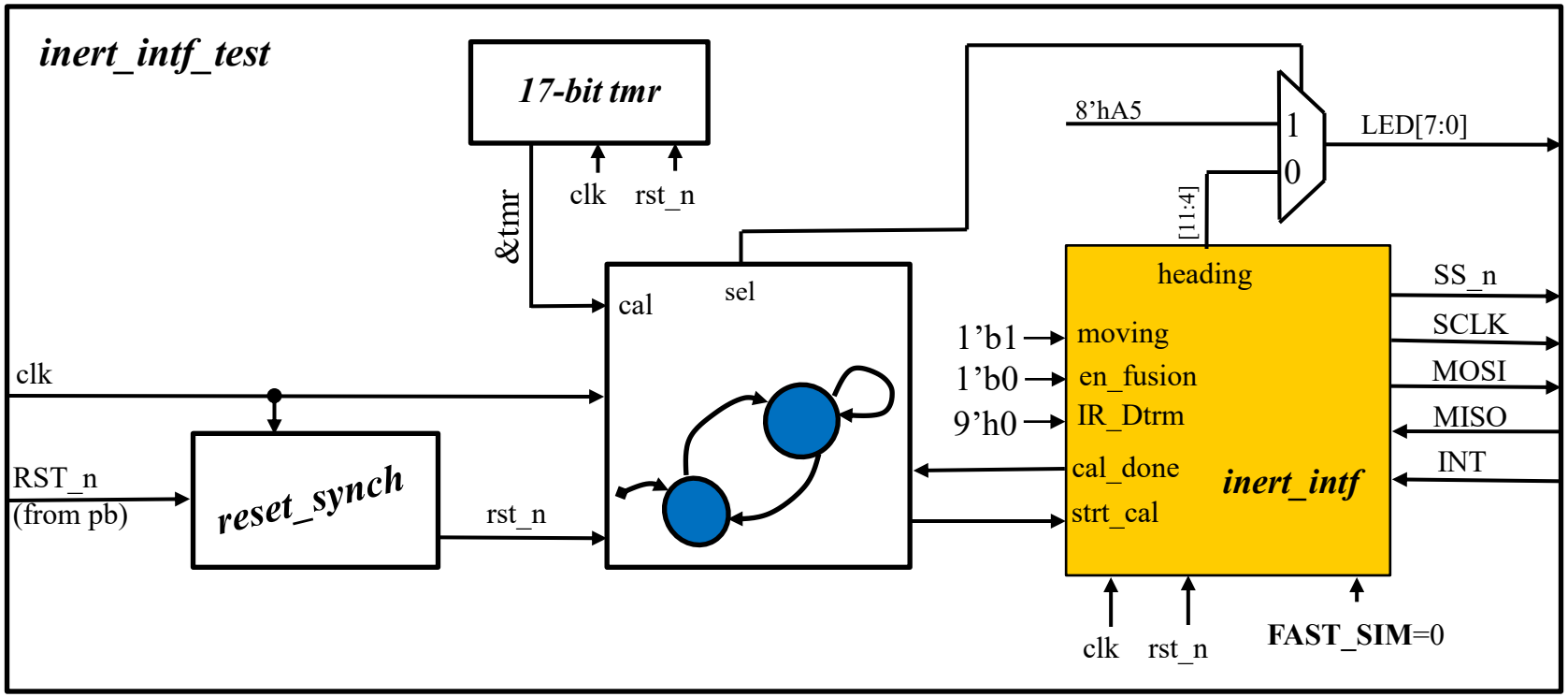
- We will map your **inert_intf.sv** to the DE0-Nano so it can be tested "for real" with an actual inertial sensor. Work in the same group as Ex17
- You will create a wrapper **inert_intf_test** that contains a few structures to help drive the inert_intf and display the results on LEDs.



Signal	Dir	Description
clk	in	50MHz clock
RST_n	in	Unsynchronized input from push button
LED[7:0]	out	Flopped upper 8-bits of heading
SPI Intf	out/ in	The SS_n, SCLK, MOSI, MISO and INT of SPI interface to 6-axis inertial sensor

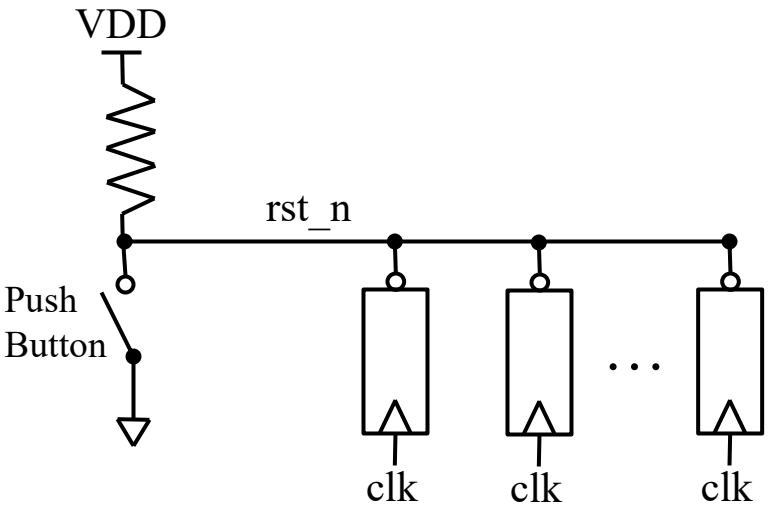
Exercise 18: Testing inert_intf on DE0

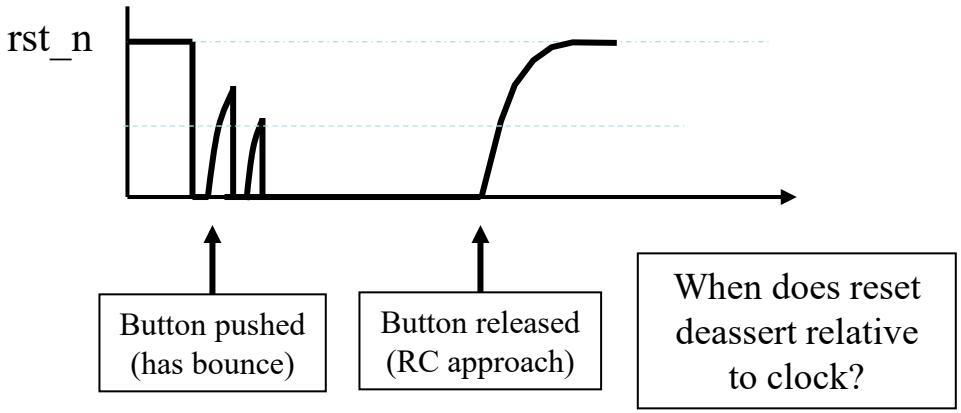
- We will map your **inert_intf.sv** to the DE0-Nano so it can be tested "for real" with an actual inertial sensor.
- You will create a wrapper **inert_intf_test** that contains a few structures to help drive the inert_intf and display the results on LEDs.



reset_synch

- On the FPGA board, we have a couple of push buttons → we will use one as the source for our async reset
- Push button is simply a switch to ground with a pull-up resistor



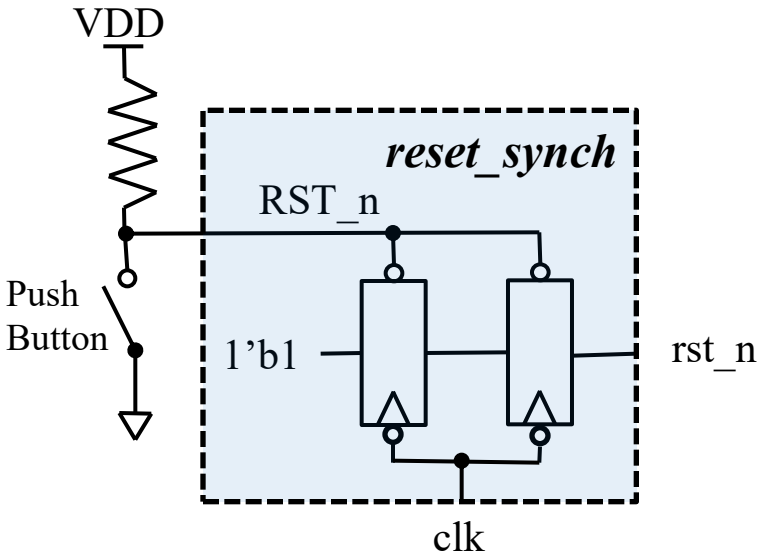


Imagine what a disaster this implementation of a global reset would be!!

reset_synch

- We want to build a reset synchronizer that takes in the raw push button signal and creates a signal that is deasserted at the negative edge of clock
- It will have the following interface:

Signal	Dir	Description
RST_n	in	Raw input from push button
clk	in	Clock, we use negative edge
rst_n	out	Our synchronized output which will form the global reset to the rest of our chip

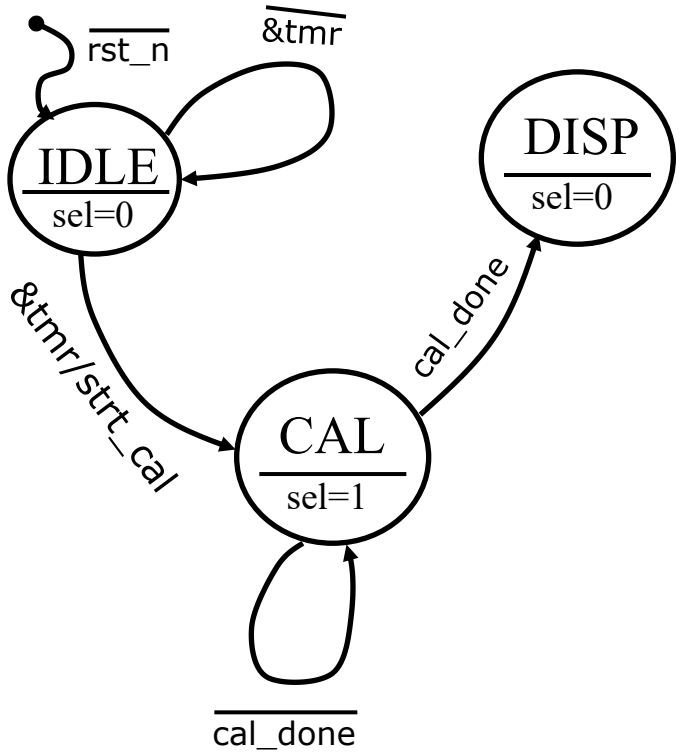


- Push of button will async reset the 2 FFs
- When button is released, we have a double flop (meta-stability reasons) to produce our global **rst_n**
- Flops are negative edge triggered so our global reset will deassert on the opposite edge of all our other flops

Code this reset synch unit (**reset_synch.sv**)

inert_intf_test State Machine

- After reset, it waits for a 17-bit free running timer to fill, then asserts **strt_cal** and waits for **cal_done**
- While in calibration, it selects 0xA5 to display on LEDs
- Once calibration is done, it displays bits [11:4] of heading on the LEDs
- Only a rest can get it out of DISP state



Exercise 18: Testing inert_intf on DE0

- Create **inert_intf_test.sv** & test it in ModelSim
- Ensure it compiles in Quartus.
  - There are Quartus project file and settings file available for download (**inert_intf_test.qpf**, **inert_intf_test.qsf**)
- Each group submits **inert_intf_test.sv, inert_intf_test_tb.sv**
  - Only 1 submission per group of 2 – remember to add 2nd group member in the submission

Demo

- Call anyone on the teaching staff over to demo on a DE0 test platform
  - Bonus credit: +0.5 if your group demos a correct implementation by Friday